

Nom et prénom: GHANMI Mohamed

Synthèse : Les attributs de qualité en architecture logicielle

Qu'est-ce qu'un attribut de qualité ?

Un attribut de qualité est une propriété mesurable d'un système logiciel. Il dit à quel point le système répond aux besoins des utilisateurs, pas seulement en termes de fonctions, mais aussi en termes de comportement général. Par exemple, le système est-il rapide ? Est-il sécurisé ? Peut-on le modifier facilement ?

La norme **ISO/IEC 25010** est le standard le plus utilisé pour décrire ces attributs. Elle définit huit grandes caractéristiques de qualité : l'adéquation fonctionnelle, la performance, la compatibilité, la facilité d'utilisation (usabilité), la fiabilité, la sécurité, la maintenabilité et la portabilité. Chacune de ces caractéristiques est ensuite découpée en sous-caractéristiques plus précises.

Pourquoi les attributs de qualité sont importants en architecture

L'architecture logicielle est la première grande décision de conception d'un système. C'est à ce moment que l'on décide comment le système sera découpé en composants et comment ces composants communiquent entre eux. Une mauvaise décision à ce stade est très difficile et coûteuse à corriger plus tard.

Les attributs de qualité jouent un rôle central dans ces décisions. Un système peut très bien faire ce qu'on lui demande fonctionnellement, mais être trop lent, trop fragile ou trop difficile à modifier. C'est pourquoi les exigences non fonctionnelles doivent être prises en compte dès le début, au même niveau que les exigences fonctionnelles.

Comment on modélise les attributs de qualité

Le modèle de qualité du système

Pour chaque caractéristique de qualité, on construit un tableau qui indique la priorité, la manière de la mesurer, et la valeur cible attendue. Par exemple : le temps de réponse à une requête doit être inférieur à 1 seconde.

Le Modèle de Qualité Fonctionnelle (FQM)

Ce modèle relie chaque fonction du système à ses exigences de qualité associées. Par exemple, dans un système bancaire mobile, la fonction "virement entre comptes" doit être fiable, sécurisée et rapide. Cela permet de ne pas oublier les contraintes de qualité liées à chaque fonction.

Le Modèle de Qualité Architecturale (AQM)

Ce modèle précise, pour chaque composant de l'architecture, comment il prend en charge les exigences de qualité. Il établit des liens de traçabilité entre les besoins du client et les choix techniques de l'architecte. Cela permet de vérifier que rien n'a été oublié.

La modélisation par scénarios (ADD)

La méthode Attribute-Driven Design propose de modéliser chaque exigence de qualité sous forme de scénario, avec six éléments : la source du stimulus (qui déclenche l'événement), le stimulus (l'événement lui-même), l'environnement (les conditions dans lesquelles cela se produit), l'artefact concerné, la réponse attendue du système, et la mesure de cette réponse. Cette approche rend les exigences concrètes et vérifiables.

Les tactiques et les patterns architecturaux

Une tactique est une décision de conception qui améliore un seul attribut de qualité. Par exemple, pour améliorer la disponibilité, on peut utiliser de la redondance active. Pour améliorer la sécurité, on peut chiffrer les données. Chaque tactique est ciblée sur un seul objectif et ne tient pas compte des compromis avec d'autres attributs.

Un pattern architectural est un ensemble de tactiques regroupées qui forment une solution connue et réutilisable, comme le modèle client-serveur ou le modèle en couches. Contrairement aux tactiques, les patterns intègrent déjà des compromis entre plusieurs attributs de qualité.

La démarche en deux niveaux

L'article de Levy, Losavio et Pollet propose une approche originale : construire d'abord une architecture de référence pour tout un domaine fonctionnel (par exemple, les systèmes de dossiers médicaux), puis l'adapter pour chaque système concret.

Au niveau du domaine, on identifie les fonctions communes à tous les systèmes, les exigences de qualité génériques, et on construit une architecture préliminaire. Au niveau du système concret, on instancie les paramètres laissés ouverts (valeurs précises de performance, politique d'authentification, niveau de disponibilité requis, etc.) et on adapte l'architecture en conséquence.

L'étude de cas illustre cela avec deux systèmes de dossiers médicaux partagés : Dopamine, un système régional avec de très fortes exigences de disponibilité et de sécurité, et Samarkand, un logiciel pour un cabinet médical avec des contraintes bien plus légères. Ces deux systèmes partagent la même architecture

de base, mais divergent significativement dans leurs composants finaux à cause de leurs attributs de qualité différents.